

Spike: Task 8**Title:** Game State Management**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike task is to demonstrate my ability to collect and analyse software performance data while applying my knowledge in comparing and evaluating functions, IDE and compiler settings.

Besides this report, what else was created?

- Code files in see C:\Users\storm\Documents\GitHub\GAMES PROGRAMMING REPO\COS30031-2023-102608927\08 - Spike - Game State Management\Zorkish\Zorkish

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- Game Specification Details Canvas Module
- OO Concepts, Patterns & Structures Canvas Modules
 - 2.1 Software Architecture of Games Lecture Slides
 - 2.2 Game States and Stages Lecture Slides
 - 3.2 Data Structures
 - 3.3 Design Patterns

Tasks undertaken:

- Planning the Zorkish Adventure
- Download and install Visual Studio
- Download and install Visual Studio C++ Debugging Packages
- Create and Configure VS Project File with a .cpp file to build.

What we found out:

Planning Phase:

Based on the game specifications details and the requirements for phase 1 I designed this UML design that visualizes the Game State management and the structuring for the world and command processing.

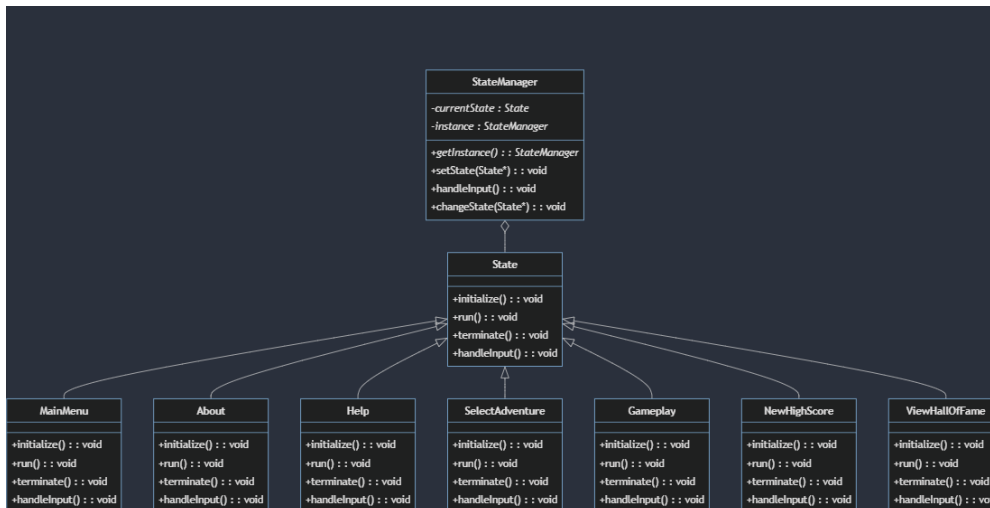


Figure 1 : Initial UML Design for Phase 1 of Zorkish Adventure Game

Developing State Class and StateManager:

```

1  #pragma once
2
3  class State {
4  public:
5      virtual void initialize() = 0; // Setting the stage
6      virtual void run() = 0;       // Running the scene
7      virtual void terminate() = 0; // Cleaning up the stage
8      virtual void handleInput() = 0;
9  };
  
```

Based on my initial UML design the State header was created to establish a clear set of methods that every state needs to initialize, run, terminate and handle inputs.

```

1  #pragma once
2  #include "State.h"
3
4  class StateManager {
5  private:
6      State* currentState;
7      static StateManager* instance;
8
9      StateManager();
10
11 public:
12     ~StateManager();
13
14     static StateManager* getInstance();
15     void changeState(State* newState);
16     void handleInput();
17 };
18

```

```

1  #include "StateManager.h"
2
3  StateManager* StateManager::instance = nullptr;
4
5  StateManager::StateManager() : currentState(nullptr) {}
6
7  StateManager::~StateManager() {
8      if (currentState) {
9          currentState->terminate();
10         delete currentState;
11     }
12 }
13
14 StateManager* StateManager::getInstance() {
15     if (!instance)
16         instance = new StateManager();
17     return instance;
18 }
19
20 void StateManager::changeState(State* newState) {
21     if (currentState) {
22         currentState->terminate();
23         delete currentState;
24     }
25     currentState = newState;
26     currentState->initialize();
27 }
28
29 void StateManager::handleInput() {
30     if (currentState) {
31         currentState->handleInput();
32     }
33 }

```

To manage these states, I created the StateManager that connects with the games main loop each state and therefore I use the currentState to manage and store the current state of the game. StateManager handles state transitions as well with methods such as setState and changeState to make sure that one instance of StateManager exists at a time by using Singleton pattern.

Developing Basic States:

When developing basic states such as MainMenu, we use the header file for MainMenu to be the blueprint that inherits from State.h so does other states with their own header file that inherits from State.h as well as we want to make sure our code follows OOP design principles.

```

1  #pragma once
2  #include "State.h"
3
4  class MainMenu : public State {
5  public:
6      void initialize() override;
7      void run() override;
8      void terminate() override;
9      void handleInput() override;
10 };
11

```

```

1  #include "MainMenu.h"
2  #include "StateManager.h"
3  #include "SelectAdventure.h"
4  #include "Help.h"
5  #include "About.h"
6  #include "ViewHallOfFame.h"
7  #include <iostream>
8
9  void MainMenu::initialize() {
10     // Display the menu as usual
11     std::cout << "Zorkish :: Main Menu" << std::endl;
12     std::cout << "-----" << std::endl;
13     std::cout << "Welcome to Zorkish Adventures" << std::endl;
14     std::cout << "1. Select Adventure and Play" << std::endl;
15     std::cout << "2. Hall Of Fame" << std::endl;
16     std::cout << "3. Help" << std::endl;
17     std::cout << "4. About" << std::endl;
18     std::cout << "5. Quit" << std::endl;
19     std::cout << "Select 1-5: ";
20 }
21
22 void MainMenu::run() {
23     //empty
24 }
25
26 void MainMenu::terminate() {
27     std::cout << "Exiting Main Menu...\n";
28 }
29
30 void MainMenu::handleInput() {
31     int input;
32     std::cin >> input;
33     switch (input) {
34     case 1:
35         StateManager::getInstance()->changeState(new SelectAdventure());
36         break;
37     case 2:
38         StateManager::getInstance()->changeState(new ViewHallOfFame());
39         break;
40     case 3:
41         StateManager::getInstance()->changeState(new Help());
42         break;
43     case 4:
44         StateManager::getInstance()->changeState(new About());
45         break;
46     case 5:
47         terminate();
48         exit(0); // Exits the program for now
49         break;
50     default:
51         std::cout << "Invalid selection. Please select 1-5: ";
52         break;
53     }
54 }
55

```

The MainMenu class utilises the initialize method to present a list of options for the user, handleInput method processes the user inputs using switch case where options are 1 to 5 that correspond to switching between states or exiting the game. Other states use similar initialization and handleInput methods that bring it back to MainMenu state.

Gameplay Phase 1 Functionality:

```

1  #include "Gameplay.h"
2  #include "StateManager.h"
3  #include "MainMenu.h"
4  #include "ViewHallOfFame.h"
5  #include <iostream>
6  #include <string>
7
8  void Gameplay::initialize() {
9     std::cout << "Welcome to Zorkish: Void World" << std::endl;
10     std::cout << "This world is simple and pointless. Used it to test Zorkish phase 1 spec." << std::endl;
11     std::cout << "> ";
12 }
13
14 void Gameplay::run() {
15     // For future phases: gameplay logic, updates, animations etc.
16 }
17
18 void Gameplay::handleInput() {
19     std::string command;
20     std::getline(std::cin, command);
21
22     if (command == "quit") {
23         std::cout << "Your adventure has ended without fame or fortune." << std::endl;
24         StateManager::getInstance()->changeState(new MainMenu());
25     }
26     else if (command == "hiscore") {
27         std::cout << "You have entered the magic word and will now see the 🎉New High Score screen." << std::endl;
28         StateManager::getInstance()->changeState(new ViewHallOfFame());
29     }
30     else {
31         std::cout << "Invalid command. Please enter a valid command." << std::endl;
32         std::cout << "> ";
33     }
34 }
35
36 void Gameplay::terminate() {
37     std::cout << "Exiting Gameplay...\n";
38 }
39

```

Since Gameplay State was mostly a placeholder and for testing purposes only. I implemented the Phase 1 requirement which didn't want us to implement anything too complex such as saving feature and Gameplay class uses enter method to set the scene as Void World for the player and only accepts commands "quit" and "hiscore" with hardcoded results.